CS498 MLsys Project Report: Reproducing and Extending Learning to Draft: Adaptive Speculative Decoding with Reinforcement Learning

Zifan Ying
zifany4@illinois.edu

Shaoshao Xiong
sx24@illinois.edu

1 Introduction

Large language model (LLM) inference is bottlenecked by autoregressive decoding, in which tokens are generated one at a time, preventing effective hardware utilization. Speculative decoding [1] addresses this by letting a smaller *draft* model propose candidate token sequences that the target model then verifies in parallel. The key trade-off is that draft quality and verification cost are interdependent: deeper drafts yield more accepted tokens per step but require more verification compute, while shallow drafts are cheap to verify but frequently rejected.

Learning to Draft (LTD) [2] is a recent ICLR 2026 paper that frames this trade-off as a reinforcement learning (RL) problem. Built on top of EAGLE-3 [3], LTD trains two separate PPO policies that at each decoding step decide (1) *how many* draft tokens to propose (the "size" controller) and (2) *how deep* to grow the draft tree before stopping (the "depth" controller). By optimizing directly for throughput, LTD achieves up to 36.4% improvement over static EAGLE-3 in greedy decoding settings.

This project aims to reproduce LTD's results, extend them to additional models and datasets not evaluated in the paper, and analyze when adaptive control provides the most benefit. This report summarizes our reproduction results and recent extensions on quantization transfer, hardware transfer, and multi-GPU deployment.

2 Background

Speculative Decoding. Speculative decoding accelerates autoregressive generation by using a lightweight draft model to propose candidate tokens and a larger target model to verify them in parallel [1, 4]. In the lossless setting, this procedure preserves the output distribution of the target model while reducing decoding latency. Beyond the original chain-based formulation, later work introduced tree-structured drafting and parallel verification, which substantially improved practical speedup [5, 6, 7].

EAGLE-3. EAGLE-3 is a strong tree-based speculative decoding framework built on the EAGLE line of work [3, 6]. Instead of verifying a single drafted chain, it constructs a draft tree and lets the target model verify multiple candidate paths in one forward pass. Its performance depends on a trade-off between draft cost and verification cost: deeper trees and larger verification sets may increase the number of accepted tokens, but they also increase latency.

Learning to Draft. Learning to Draft (LTD) formulates speculative decoding as a reinforcement learning problem and replaces EAGLE-3's static control strategy with two adaptive policies [2]. Importantly, the two policies control *draft depth* and *verification size*, respectively. The depth policy decides whether to continue expanding the draft tree after each draft-model forward pass, while the size policy selects how many candidate tokens from the generated draft tree should be sent to the target model for verification. Both policies are optimized with PPO using throughput as the reward, directly balancing acceptance length against wall-clock latency [8, 9].

The core claim of LTD is that speculative decoding should optimize throughput rather than acceptance length alone. The paper reports speedup ratios ranging from $2.24\times$ to $4.32\times$, and up to 36.4% improvement over a strong EAGLE-3 baseline in greedy decoding settings [2]. Our project reproduces this idea on five target models and extends the evaluation to a broader set of datasets than those emphasized in the original main results.

3 Implementation

RL Training. We implement the size and depth controllers as PPO agents using Stable-Baselines3 [9]. Training is performed on a single GPU using the EAGLE-3 inference loop as the environment. At each RL timestep, the agent observes a feature vector and takes a discrete action.

The *size policy* uses a 1268-dimensional observation vector comprising the EA layer's candidate confidence scores (1210 floats), current sequence length (29 floats), and a step counter (29 floats). Its action space is the draft

token budget. The policy network has hidden layers of size $[1024, 256]$ and is trained for 100 000 PPO timesteps with learning rate 10^{-3}, discount $\gamma = 0.9$, entropy coefficient 0.01, and batch size 256.

The *depth policy* uses a 128-dimensional observation vector comprising per-level confidence scores (100 floats), context-length features (14 floats), and a step counter (14 floats). It is trained as a binary stop/continue classifier for 1 000 000 timesteps with $\gamma = 0.999$ and a single hidden layer of size 1024.

Models. We train and evaluate controllers for five target models spanning different model families and scales:

- **Llama-3.1-8B-Instruct** with EAGLE3-LLaMA3.1-Instruct-8B
- **DeepSeek-R1-Distill-Llama-8B** with EAGLE3-DeepSeek-R1-Distill-LLaMA-8B
- **Qwen3-14B** with Qwen3-14B_eagle3
- **Qwen3-32B** with Qwen3-32B_eagle3 under 8-bit quantization
- **Vicuna-13B-v1.3** with EAGLE3-Vicuna1.3-13B

Each model has its own size and depth policy checkpoint trained from scratch.

Datasets. We train controllers on the HumanEval coding dataset. For benchmarking, we evaluate on 11 datasets covering diverse task types: *alpaca* (instruction following), *gsm8k* (grade-school math), *humaneval* (code generation), *mt_bench* (multi-turn dialog), *qa* (question answering), *sum* (summarization), *math* (competition math), *leetcode* (algorithmic coding), *mmlu* (multi-subject knowledge), *bbh* (big-bench hard), and *theoremqa* (theorem proving). This extends the LTD paper, which evaluated on a subset of these tasks.

4 Evaluation Setup

We use greedy decoding (temperature 0.0) to match the primary setting emphasized in the LTD paper [2]. The initial reproduction experiments run on a single NVIDIA GPU with CUDA and use the same 11-dataset benchmark suite. The new Qwen3-32B reproduction runs on an H100 using 8-bit quantization with 20 samples per dataset, while the quantization-transfer and hardware-transfer sweeps use 50 samples on four representative datasets: HumanEval, Alpaca, GSM8K, and MT-Bench. All runs cap generation at 256 output tokens.

For transfer experiments, we change only the deployment condition while reusing trained policies. Quantization transfer evaluates Qwen3-14B policies trained under bf16, int4, and int8 against all three evaluation precisions. Hardware transfer evaluates bf16 Qwen3-14B policies trained on H100 or A5090 when deployed on an RTX A6000. Finally, the multi-GPU experiment evaluates Qwen3-32B under naive model parallelism with HuggingFace's automatic device map, which shards layers across two GPUs without adding pipeline overlap.

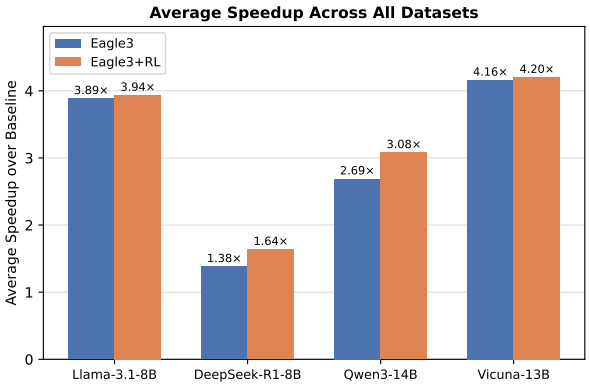

Figure 1: Average speedup over the autoregressive baseline across all 11 datasets for the initial four-model reproduction. Each pair of bars shows Eagle3 (blue) and Eagle3+RL (orange) for one model. DeepSeek-R1-8B shows the largest *relative* gain; Qwen3-14B shows the largest *absolute* gain.

We evaluate on 11 datasets spanning instruction following, reasoning, coding, QA, summarization, and broad knowledge: Alpaca [10], GSM8K [11], HumanEval [12], MT-Bench [13], Natural Questions-style QA [14], MMLU [15], BBH [16], and several additional datasets used in our extension experiments.

We report three metrics:

- **Throughput** (tokens/second): generated tokens divided by wall-clock time.
- **Speedup**: ratio of method throughput to autoregressive baseline throughput.
- **Avg. acceptance length**: mean number of accepted tokens per speculative step.

We compare three configurations: (1) *Baseline*, standard autoregressive decoding with the target model; (2) *Eagle3*, static EAGLE-3; and (3) *Eagle3+RL*, EAGLE-3 augmented with the LTD-style adaptive size and depth controllers trained in this work.

5 Results

Overall speedup. Table 1 summarizes the average speedup across all 11 datasets for the initial four-model reproduction. Eagle3+RL improves over static Eagle3 for all four models. The gains, however, are highly model-dependent: they are small for Llama-3.1-8B and Vicuna-13B, but much larger for DeepSeek-R1-Distill-Llama-8B

Table 1: Average speedup over autoregressive baseline across all 11 datasets for the initial four reproduced models. Eagle3+RL uses the LTD-style adaptive controllers trained in this work.

Model	Eagle3	Eagle3+RL
Llama-3.1-8B-Instruct	3.89×	**3.94**×
DeepSeek-R1-Distill-Llama-8B	1.38×	**1.64**×
Qwen3-14B	2.69×	**3.08**×
Vicuna-13B-v1.3	4.16×	**4.20**×

Table 2: Per-dataset speedup for Qwen3-14B. Eagle3+RL consistently outperforms Eagle3 across all task categories.

Dataset	Eagle3	Eagle3+RL
alpaca	2.51×	2.93×
gsm8k	3.05×	3.42×
humaneval	2.73×	3.19×
mt_bench	2.63×	3.03×
qa	2.30×	2.68×
sum	2.18×	2.55×
math	2.99×	3.35×
leetcode	2.78×	3.21×
mmlu	2.79×	3.18×
bbh	2.78×	3.17×
theoremqa	2.83×	3.19×

and Qwen3-14B. This matches the central LTD hypothesis that adaptive control is most useful when the best speculative-decoding budget varies substantially across contexts [2].

Llama-3.1-8B-Instruct. Eagle3 already achieves strong speedups of 3.28×–4.59× on this model, and the RL controller adds only a modest average gain from 3.89× to 3.94×. Static acceptance lengths are already high, and in several datasets the RL controller changes acceptance length only marginally. This suggests that the Llama-3.1 draft/verification pair is already close to a saturated operating point under static Eagle3 settings.

DeepSeek-R1-Distill-Llama-8B. DeepSeek-R1-Distill-Llama-8B shows the largest *relative* gain from adaptive control: the average speedup increases from 1.38× to 1.64×, corresponding to roughly 18.8% relative improvement. Its static acceptance lengths are much lower than those of Llama and Vicuna, indicating that speculative verification is harder on this reasoning-oriented model. In this regime, adaptive control is particularly useful because it can avoid wasting compute on low-value draft expansions.

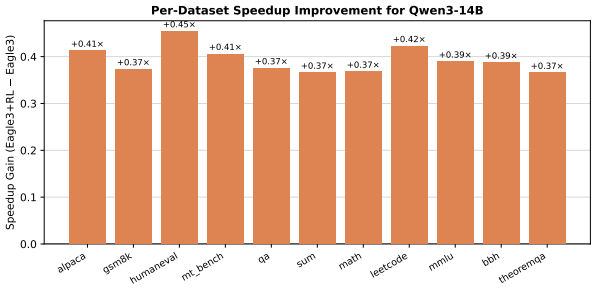

Figure 2: Per-dataset speedup gain (Eagle3+RL − Eagle3) for Qwen3-14B. All 11 datasets show a positive improvement, confirming that the controller trained on HumanEval transfers across diverse task types.

Table 3: Qwen3-32B reproduction under 8-bit quantization on H100. RL/Eagle3 reports the throughput ratio of the learned controller relative to static Eagle3.

Dataset	Eagle3	Eagle3+RL	RL/Eagle3
alpaca	2.99×	3.19×	1.069
gsm8k	3.29×	3.45×	1.047
humaneval	2.93×	3.14×	1.070
mt_bench	2.77×	2.91×	1.048
qa	2.54×	2.71×	1.066
sum	2.49×	2.46×	0.992
math	3.36×	3.60×	1.070
leetcode	2.95×	3.12×	1.058
mmlu	3.31×	3.47×	1.046
bbh	3.16×	3.30×	1.045
theoremqa	3.15×	3.41×	1.083
Average	3.00×	**3.16**×	**1.054**

Qwen3-14B. Qwen3-14B shows the largest *absolute* average gain among the reproduced models, improving from 2.69× to 3.08× on average. Table 2 shows that the improvement is consistent across all 11 datasets, with gains ranging from about +0.37× to +0.46×. This stability is notable because the controllers were trained on HumanEval but still transfer well to dialogue, QA, summarization, and knowledge-heavy benchmarks.

Figure 2 plots the per-dataset speedup gain (Eagle3+RL − Eagle3) for Qwen3-14B. All 11 bars are positive and the gains are roughly uniform (0.37–0.46×), providing strong evidence that the learned controller generalises well beyond its HumanEval training distribution.

Qwen3-32B under 8-bit quantization. The new Qwen3-32B experiment completes the five-model reproduction. Table 3 shows that static Eagle3 reaches 3.00× average speedup over autoregressive decoding, while Eagle3+RL increases this to 3.16×, a 5.4% throughput gain

over static Eagle3. The gain is positive on 10 of 11 datasets; summarization is the only regression, where the learned controller is slightly slower than static Eagle3 (2.46× vs. 2.49×). The average acceptance length also increases from 3.72 to 3.98 tokens per speculative step. Compared with Qwen3-14B, the 32B model still benefits from adaptive control, but the margin is smaller, suggesting that the 8-bit Qwen3-32B setup is closer to a saturated static operating point.

Vicuna-13B-v1.3. Vicuna-13B achieves the highest static Eagle3 speedups in our experiments, and the average improvement from RL is correspondingly small (4.16× → 4.20×). The depth controller almost always selects the maximum allowed depth, suggesting that for this model the learned policy is close to an "always continue" strategy. Moreover, the benefit of RL is uneven across datasets: it improves substantially on HumanEval and slightly on Math and LeetCode, but is neutral or slightly worse on QA, summarization, MMLU, and TheoremQA. Therefore, the Vicuna results should be interpreted as evidence that adaptive control is not uniformly beneficial once the static baseline is already very strong.

Acceptance length vs. speedup. Eagle3+RL does not always increase the average acceptance length relative to static Eagle3. This is consistent with the LTD paper's main argument: the controller is optimized for throughput rather than acceptance length itself [2]. In several cases, a similar or even slightly lower acceptance length still yields better end-to-end speedup because the controller avoids expensive draft/verification choices whose latency cost outweighs their benefit.

Figure 3 makes this concrete for the initial four-model reproduction. Each point represents one model–dataset pair, with the horizontal axis showing the change in average acceptance length and the vertical axis showing the change in speedup. Many points lie in the lower-right quadrant: speedup improves despite acceptance length decreasing. This confirms that throughput-oriented optimization departs meaningfully from acceptance-length maximization.

6 Transfer and Deployment Extensions

Quantization transferability. We next test whether a Qwen3-14B policy trained under one precision remains effective under another. Table 4 reports the average Eagle3+RL throughput divided by static Eagle3 throughput across HumanEval, Alpaca, GSM8K, and MT-Bench. Across all 36 policy–precision–dataset combinations, Eagle3+RL averages 1.056× over static Eagle3, and only two individual runs fall below 1.0×. This indicates that

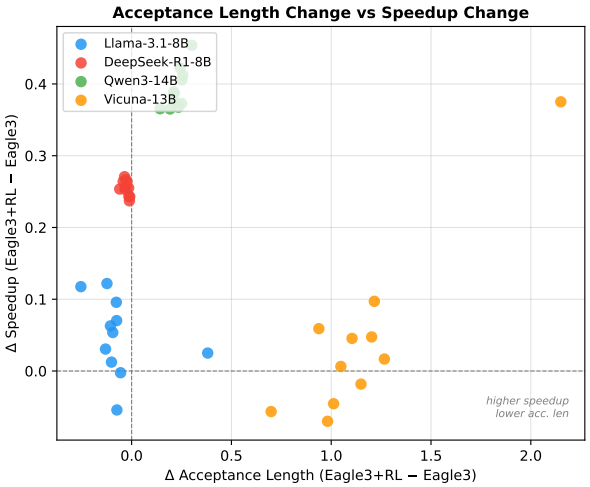

Figure 3: Each point is one model–dataset pair from the initial four-model reproduction. Points in the lower-right quadrant (positive ΔSpeedup, negative ΔAcceptance Length) show that the RL controller can improve throughput even without increasing accepted tokens per step, consistent with LTD's throughput-first objective.

Table 4: Quantization transfer on Qwen3-14B. Entries are average Eagle3+RL / Eagle3 throughput ratios across four datasets.

Policy	bf16	int4	int8	Avg.
bf16	1.046	1.056	1.069	1.057
int4	**1.118**	1.060	1.057	**1.078**
int8	1.018	1.005	**1.074**	1.032

LTD policies are mostly transferable across bf16, int4, and int8 deployment precisions.

The strongest transfer comes from the int4-trained policy, which averages 1.078× over static Eagle3 and even improves bf16 evaluation by 11.8%. The int8-trained policy is less robust: it is best when evaluated under int8, but it nearly matches static Eagle3 under int4 and has two dataset-level regressions. The policy action statistics explain part of this difference. The int4 policy uses a smaller verification budget (120 tokens) and shallower depth, while the int8 policy uses a larger budget (210 tokens) and deeper trees. The more aggressive int8 policy appears more sensitive to changes in precision-specific latency and acceptance behavior.

Hardware transferability. Hardware transfer is much weaker. Table 5 evaluates bf16 Qwen3-14B policies trained on A5090 or H100 when deployed on an RTX A6000. Both transferred policies remain faster than the autoregressive baseline, but both are substantially slower than static Eagle3 on the target GPU. The A5090-trained

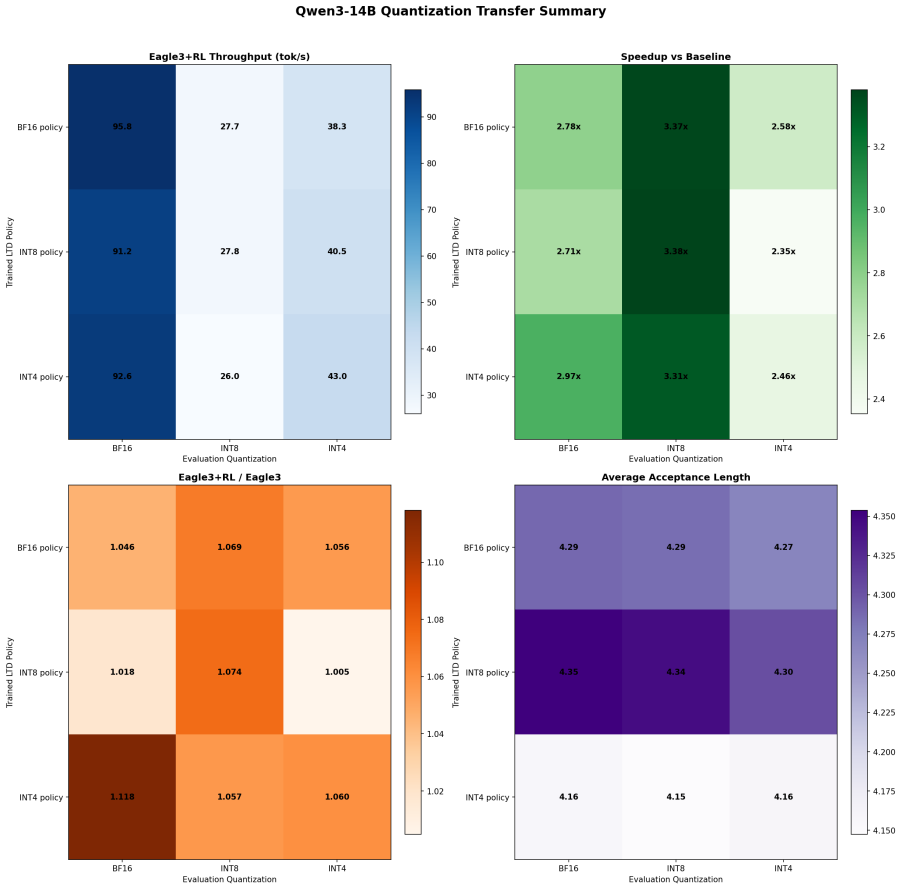

Figure 4: Quantization-transfer summary for Qwen3-14B. The learned policies generally remain faster than static Eagle3 across deployment precisions, but the int8-trained policy is the least robust when moved to other precisions.

Table 5: Hardware transfer on Qwen3-14B bf16. Policies trained on A5090 or H100 are evaluated on RTX A6000.

Train	E3 sp.	RL sp.	RL/E3	Size	Depth
A5090	2.55×	1.76×	0.693	220	4.09
H100	2.55×	1.89×	0.739	190	5.21

policy averages only 0.693× of static Eagle3 throughput, while the H100-trained policy reaches 0.739×.

This result is the clearest evidence that LTD's throughput reward is hardware-specific. The learned policy internalizes the latency trade-off of the training GPU: how expensive extra draft expansion is, how well larger verification batches are amortized, and where memory bandwidth becomes limiting. When moved to an RTX A6000, those trade-offs change enough that the learned actions become worse than the static Eagle3 heuristic.

Naive two-GPU model parallelism. Finally, we evaluate whether LTD remains effective when Qwen3-32B is split across two GPUs using naive layer sharding. Table 6 compares the single-GPU and two-GPU Qwen3-32B 8-bit runs. The relative speedups are almost unchanged: Eagle3+RL is 1.054× over static Eagle3 on one GPU and 1.053× under the two-GPU device map. However, absolute throughput drops by about 8% for all methods under two-GPU sharding.

Table 6: Qwen3-32B 8-bit under single-GPU and naive two-GPU model parallelism. Throughput is averaged across all 11 datasets.

Setting	Base tp	E3 tp	RL tp	RL sp.	RL/E3
1 GPU	5.60	16.75	17.67	3.16×	1.054
2 GPUs	5.11	15.36	16.19	3.17×	1.053

Thus, naive model parallelism does not provide a throughput win in this setting; it mainly makes the model easier to place in memory. Still, the learned LTD controller remains effective under sharding because the relative speculative-decoding trade-off is preserved. A more promising direction would be true pipeline or tensor parallel speculative decoding, where multiple GPUs reduce

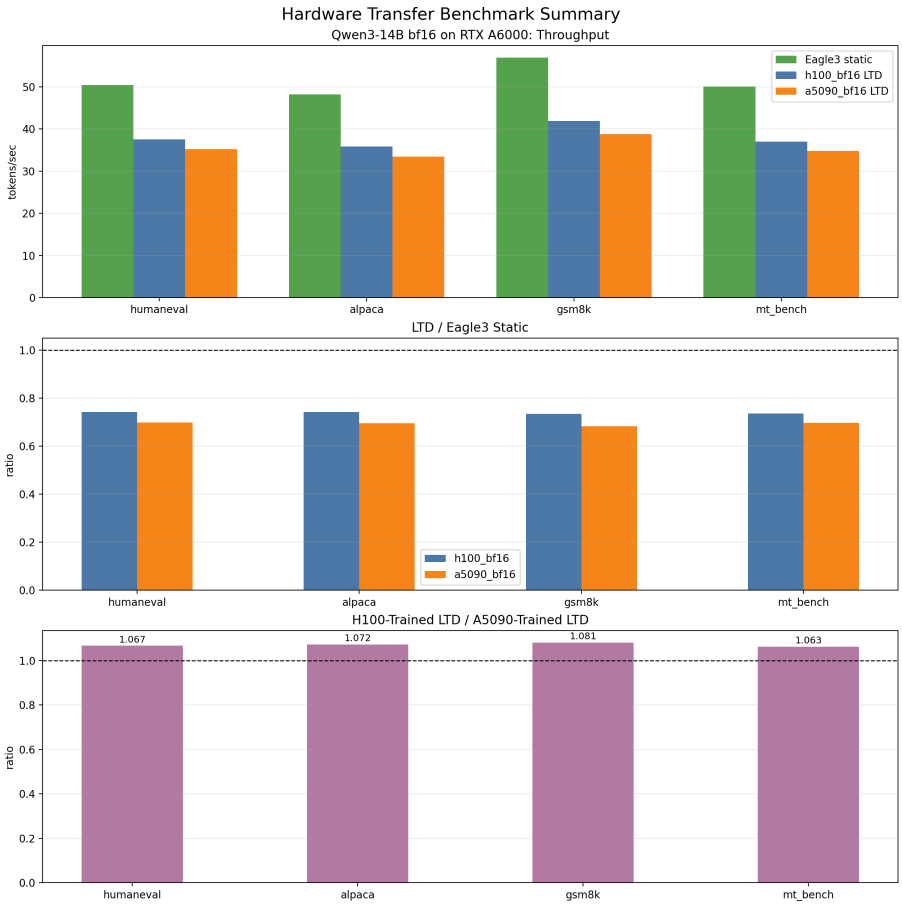

Figure 5: Hardware-transfer summary for Qwen3-14B bf16. Transferred policies still beat autoregressive decoding, but they fail to beat static Eagle3 on RTX A6000.

verification latency rather than only partitioning model layers.

7 Discussion and Future Work

Progress summary. We have successfully reproduced the core LTD pipeline on five target models and verified its main qualitative conclusion: reinforcement-learning-based adaptive control improves throughput over static EAGLE-3 in most same-hardware, same-precision settings. Relative to the original paper, our extensions now include broader dataset coverage, Qwen3-32B under 8-bit quantization, quantization-transfer sweeps, hardware-transfer sweeps, and a naive two-GPU deployment study.

When does adaptive control help most? Our results suggest that adaptive control helps most when the static Eagle3 operating point is far from optimal and the deployment condition matches training. This is especially clear for DeepSeek-R1-Distill-Llama-8B and Qwen3-14B, where the average gains are substantial. By contrast, when static Eagle3 already performs very well, as in Llama-3.1-8B and Vicuna-13B, the learned controller has much less room to improve and may only help on specific task categories. Qwen3-32B falls between these cases: it benefits consistently under 8-bit quantization, but the average gain is only 5.4%.

Figure 6 plots each model–dataset pair from the initial reproduction with static Eagle3 speedup on the horizontal axis and the RL gain (Eagle3+RL − Eagle3) on the vertical axis. The negative trend line confirms the saturation hypothesis quantitatively: as the static baseline approaches its ceiling, the marginal benefit of adaptive control diminishes.

Limitations. The current experiments only cover greedy decoding. The original LTD paper also studies temperature-1 sampling and reports that adaptive control remains effective in that setting [2]. In addition, our policies are trained only on HumanEval, so the current results do not isolate how much of the observed transfer comes from the policy design itself versus the specific training

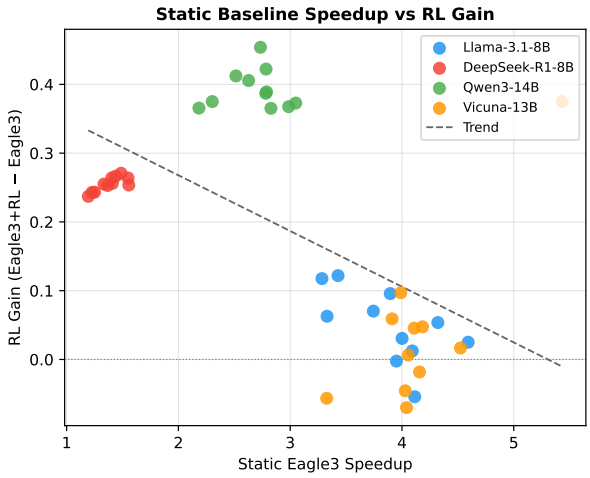

Figure 6: Static Eagle3 speedup versus RL gain for every model–dataset pair in the initial reproduction. The dashed trend line shows a negative correlation: stronger static baselines leave less room for the RL controller to improve, validating the saturation hypothesis.

distribution. The transfer experiments are also smaller than the main reproduction: quantization and hardware transfer use four datasets, and hardware transfer only tests deployment on one target GPU. Finally, the two-GPU experiment uses naive layer sharding, not communication-aware pipeline or tensor parallelism.

Next steps. Useful next steps are:

- evaluate under sampling (temperature > 0) and compare against the paper's reported trends;
- study alternative reward functions, such as acceptance-length-only and latency-aware objectives;
- analyze controller decisions more directly through distributions of selected size and depth actions;
- investigate whether hardware-conditioned features or lightweight target-GPU fine-tuning can improve hardware transfer;
- explore true distributed speculative decoding, where drafting and verification are pipelined or tensor-parallelized across GPUs to measure whether overlap can outweigh communication overhead.

References

[1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*, 2023.

[2] Jiebin Zhang, Zhenghan Yu, Liang Wang, Nan Yang, Eugene J. Yu, Zheng Li, Yifan Song, Dawei Zhu, Xingxing Zhang, Furu Wei, and Sujian Li. Learning to draft: Adaptive speculative decoding with reinforcement learning, 2026.

[3] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scalable speculative decoding with relaxed draft-token consistency. *arXiv preprint arXiv:2503.01840*, 2025.

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[5] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2023.

[6] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432, 2024.

[7] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, pages 5209–5235, 2024.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[9] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[10] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*, 2023.

[11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro

Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

[14] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

[15] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[16] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Aditya Abid, Adam Fisch, Adam R Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.

A Full Per-Dataset Statistics

Table 7 reports per-dataset acceptance length, draft size, depth stop, and speedup for the initial four-model reproduction. Several findings from the main text are directly visible here: (1) Vicuna-13B consistently reaches Depth-Stop = 12.00, confirming the depth controller defaults to "always continue"; (2) DeepSeek-R1-8B has acceptance lengths below 2.5 across all datasets, explaining why static Eagle3 yields only modest speedups; (3) Qwen3-14B gains are uniformly positive and spread across task types, supporting cross-task generalisation.

Table 7: Per-model, per-dataset statistics for Eagle3 and Eagle3+RL. Acc. Len = average accepted tokens per speculative step; SizeTok = average number of draft tokens selected by the size policy; DepthStop = average tree depth at which the depth policy halts; Speedup = throughput ratio relative to the autoregressive baseline.

	Eagle3		**Eagle3+RL**			
Dataset	**Acc. Len**	**Speedup**	**Acc. Len**	**SizeTok**	**DepthStop**	**Speedup**
Llama-3.1-8B-Instruct						
Alpaca	6.96	4.32×	6.86	49.4	7.22	4.37×
GSM8K	6.47	4.09×	6.37	50.0	7.10	4.10×
HumanEval	7.23	4.59×	7.61	50.0	8.58	4.62×
MT-Bench	6.29	4.00×	6.16	50.0	6.74	4.03×
QA	5.22	3.28×	4.97	50.0	5.56	3.40×
Sum	5.40	3.43×	5.28	50.0	6.02	3.55×
Math	6.22	3.95×	6.16	50.0	7.26	3.95×
LeetCode	6.44	4.11×	6.37	50.0	7.52	4.06×
MMLU	5.83	3.74×	5.75	50.0	6.65	3.81×
BBH	6.07	3.90×	5.99	50.0	6.50	3.99×
TheoremQA	5.17	3.33×	5.06	50.0	6.49	3.39×
DeepSeek-R1-Distill-Llama-8B						
Alpaca	1.89	1.23×	1.88	60.0	3.04	1.47×
GSM8K	2.40	1.56×	2.34	60.0	3.15	1.81×
HumanEval	2.29	1.49×	2.26	60.0	3.07	1.76×
MT-Bench	2.11	1.37×	2.07	60.0	3.08	1.62×
QA	1.84	1.19×	1.82	60.0	3.03	1.43×
Sum	1.93	1.25×	1.92	60.0	3.02	1.49×
Math	2.39	1.55×	2.35	60.0	3.17	1.81×
LeetCode	2.16	1.40×	2.13	60.0	3.06	1.67×
MMLU	2.06	1.33×	2.04	60.0	3.08	1.59×
BBH	2.22	1.44×	2.19	60.0	3.09	1.70×
TheoremQA	2.17	1.41×	2.14	60.0	3.10	1.67×
Qwen3-14B						
Alpaca	3.59	2.51×	3.85	220.0	3.89	2.93×
GSM8K	4.36	3.05×	4.61	220.0	4.55	3.42×
HumanEval	3.90	2.73×	4.21	220.0	3.98	3.19×
MT-Bench	3.75	2.63×	4.00	220.0	3.91	3.03×
QA	3.28	2.30×	3.49	220.0	3.75	2.68×
Sum	3.14	2.18×	3.28	220.0	3.16	2.55×
Math	4.27	2.99×	4.50	220.0	4.43	3.35×
LeetCode	3.97	2.78×	4.22	220.0	3.93	3.21×
MMLU	3.98	2.79×	4.19	220.0	4.03	3.18×
BBH	3.97	2.78×	4.18	220.0	4.03	3.17×
TheoremQA	4.04	2.83×	4.24	220.0	4.16	3.19×
Vicuna-13B-v1.3						
Alpaca	6.23	3.91×	7.17	200.0	12.00	3.97×
GSM8K	6.25	4.06×	7.30	200.0	12.00	4.06×
HumanEval	8.35	5.43×	10.50	200.0	12.00	5.81×
MT-Bench	6.42	4.11×	7.53	197.5	11.85	4.15×
QA	5.18	3.33×	5.88	200.0	12.00	3.27×
Sum	6.66	4.16×	7.81	200.0	12.00	4.14×
Math	6.16	3.99×	7.38	200.0	12.00	4.09×
LeetCode	6.89	4.52×	8.15	200.0	12.00	4.54×
MMLU	6.14	4.03×	7.15	200.0	12.00	3.98×
BBH	6.41	4.18×	7.61	200.0	12.00	4.23×
TheoremQA	6.15	4.04×	7.13	200.0	12.00	3.97×